

Pilot MVP — Affiliate Referral System for Used Car Sale

Context (read before coding)

This is a **pilot MVP** to sell one car (Renault Clio V 2022) using an affiliate referral model. The seller (owner) keeps the car; affiliates promote it; buyers redeem a unique code to get a discount; affiliate gets a fixed commission on closed sales.

Hard constraint: 3 hours from zero to deployed on Vercel. If a feature can't fit in that window, it goes to "Future" and ships as v2.

This is for ONE car only. Do not generalize into a multi-listing platform yet. Hardcode the car details. We'll refactor when the pilot validates the model.

Tech stack (locked — do not deviate)

- **Next.js 14+** (App Router, TypeScript)
 - **Tailwind CSS** for styling, **shadcn/ui** for components
 - **Supabase** (Postgres + free tier) for database
 - **Vercel** for hosting (auto-deploy from `main`)
 - **WhatsApp notifications:** import existing module from owner's other repo (see "External Module" below)
 - **No auth library.** Admin route protected by a single env var password via middleware.
 - **No email sending in this MVP.** WhatsApp only.
-

Pricing model (the math the system encodes)

- **Owner net target:** €15,000 (fixed)
- **List price** (shown everywhere publicly): €15,800
- **Affiliate commission** (fixed per closed sale): €500
- **Buyer discount** (when code is redeemed): €300
- **Buyer pays with code:** €15,500

These four numbers live in environment variables so they can be tuned without redeploying:

- `NEXT_PUBLIC_LIST_PRICE=15800`
 - `OWNER_NET_TARGET=15000`
 - `AFFILIATE_COMMISSION=500`
 - `BUYER_DISCOUNT=300`
-

Database schema (Supabase)

affiliates

column	type	notes
id	uuid pk	default <code>gen_random_uuid()</code>
name	text not null	
email	text not null	
whatsapp	text not null	E.164 format, validate on input
ref_code	text unique not null	auto-generated, format: <code>{name-slug}-{4 random alphanum}</code> e.g. <code>pedro-x7k2</code>
created_at	timestamp tz	default <code>now()</code>

leads

column	type	notes
id	uuid pk	default <code>gen_random_uuid()</code>
affiliate_id	uuid fk → affiliates.id	nullable (direct leads with no referrer)
buyer_name	text not null	
buyer_whatsapp	text not null	E.164
discount_code	text unique not null	6-char uppercase alphanumeric, e.g. <code>K7M2QX</code>
status	text not null	enum-as-text: <code>new</code> <code>contacted</code> <code>test_drive</code> <code>closed_won</code> <code>closed_lost</code>
notes	text	nullable, admin-editable
created_at	timestamp tz	default <code>now()</code>
updated_at	timestamp tz	default <code>now()</code> , auto-update on change

Row Level Security

Enable RLS on both tables. For this MVP all writes go through server actions using the service role key — public anon key has zero permissions. Don't expose the service role key to the client.

Routes

`/` — Public landing page (the car)

- Hardcoded content for the Clio V 2022: photos, year, km, options, list price `€15,800`
- Hero CTA button: "Tenho interesse" → `/interesse`
- If URL contains `?ref=xxx`, store the ref code in a httpOnly cookie (`affiliate_ref`, 30-day expiry) **server-side via middleware or a route handler**, then strip the param and serve the page normally
- Footer: small text "Anúncio particular. Não há comissão para o comprador." (private listing, no commission charged to buyer)
- Copy in **Portuguese (PT-PT)**. Draft copy at the bottom of this spec.

`/interesse` — Buyer lead form

Fields:

- Nome completo (text, required)
- WhatsApp (text, required, validate as PT phone or E.164)
- Implicit hidden: `ref_code` from cookie (if present)

On submit (server action):

1. Validate inputs server-side. If invalid, return field errors.
2. Look up affiliate by `ref_code` cookie if present.
3. Generate `discount_code` (6 uppercase alphanumeric, retry on collision, max 5 retries).
4. Insert `leads` row.
5. Trigger two WhatsApp messages via the external module:
 - To buyer: confirmation + their unique code + owner's WhatsApp link
 - To owner: new lead notification with buyer name, WhatsApp, code, affiliate name (or "direto")
6. Redirect to `/interesse/obrigado?code=XXXXXX`.

`/interesse/obrigado` — Confirmation page

- Shows the buyer's unique code prominently
- Shows: "Recebemos os seus dados. O proprietário vai contactá-lo por WhatsApp."

- Button: "Falar agora por WhatsApp" → opens `https://wa.me/{OWNER_WHATSAPP}?text=Olá! Tenho o código {code} e tenho interesse no Clio.`

`/afiliado` — Affiliate signup

Fields:

- Nome
 - Email
 - WhatsApp On submit (server action):
1. Validate.
 2. Generate `ref_code`: slugify(name first word, lowercase, ascii-only) + `-` + 4 random lowercase alphanumeric. Retry on collision.
 3. Insert `affiliates` row.
 4. Redirect to `/afiliado/{ref_code}`.

`/afiliado/[ref_code]` — Affiliate dashboard (semi-public)

- Looks up affiliate by `ref_code`. If not found, 404.
- Shows:
 - Their unique referral URL: `{BASE_URL}/?ref={ref_code}` with a "Copiar link" button
 - Their commission per closed sale: `€{AFFILIATE_COMMISSION}`
 - Counts: total leads, leads by status
 - List of their leads (buyer first name only for privacy — e.g. "Maria S." — plus date and status)
- No auth — security through obscurity of the `ref_code` URL. Acceptable for pilot.

`/admin` — Owner admin

Protected by middleware: requires `Authorization: Basic` header or a cookie set via a `/admin/login` route that checks `ADMIN_PASSWORD`. Simplest: HTTP Basic Auth via middleware checking env var.

Shows:

- **Leads tab:** table of all leads with buyer name, WhatsApp, affiliate name, code, status (editable dropdown), `created_at`, notes (editable). Status change saves immediately via server action.
- **Affiliates tab:** table of all affiliates with name, `ref_code`, lead count, won count, commission owed (`won_count × AFFILIATE_COMMISSION`).
- **Pay tracking:** on each affiliate row, a "Marcar como pago" toggle (just a boolean in a new column `commission_paid` on leads, or a separate `payouts` table). For 3h MVP: add `commission_paid boolean default false` to `leads`, checkbox in admin to flip it.

When a lead's status changes to `closed_won`, the admin UI surfaces: "Pagar €500 a {affiliate name} via WhatsApp {affiliate whatsapp}". No automation of payment.

External module: WhatsApp notifications

The owner will provide their existing WhatsApp send module from another repo (likely a thin wrapper around the WhatsApp Business Cloud API or a third-party like Twilio).

Expected interface (Claude Code: adapt to whatever the owner's repo actually exposes):

```
ts

// lib/whatsapp.ts - imported/adapted from owner's repo
export async function sendWhatsApp(to: string, message: string): Promise<void>;
```

Two templates (define as functions in `lib/messages.ts`):

```
ts

export function buyerConfirmation(opts: { buyerName: string; code: string; ownerWhat: string }) {
  // "Olá {buyerName}! Recebemos o seu interesse no Clio V 2022. O seu código de desco

export function ownerNewLead(opts: { buyerName: string; buyerWhatsapp: string; code: string }) {
  // "🚗 Novo lead Clio: {buyerName} ({buyerWhatsapp}). Código: {code}. Afiliado: {aff
```

If the WhatsApp module is not yet integrated at deploy time, **fail gracefully**: log the message to console, save lead anyway, do not block submission. Add an env var `WHATSAPP_ENABLED=false` to toggle this.

Environment variables

```
# Database
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
NEXT_PUBLIC_SUPABASE_ANON_KEY=

# Pricing (public-safe)
NEXT_PUBLIC_LIST_PRICE=15800
OWNER_NET_TARGET=15000
AFFILIATE_COMMISSION=500
BUYER_DISCOUNT=300

# Owner contact
OWNER_WHATSAPP=351XXXXXXXXX # E.164 no plus sign for wa.me links
```

```
NEXT_PUBLIC_OWNER_FIRST_NAME=  # for personalization

# Admin
ADMIN_PASSWORD=                # strong, set in Vercel only

# WhatsApp module (depends on owner's existing impl)
WHATSAPP_ENABLED=true
WHATSAPP_API_KEY=              # or whatever the existing module needs

# Site
NEXT_PUBLIC_BASE_URL=https://...vercel.app
```

Out of scope (do NOT build in this 3h window)

- Multi-car listings / generalized seller onboarding
- Payment processing / Stripe / automated payouts
- Email notifications
- SMS fallback
- OAuth / user accounts beyond admin password
- Affiliate ranking, leaderboards, gamification
- Multi-language (PT-PT only for now)
- Analytics dashboards beyond the basic counts
- Mobile app
- Image uploads (car photos are committed to the repo as static assets)
- Audit log / change history
- GDPR cookie banner (defer — for pilot, add a one-line privacy notice on forms; full RGPD compliance before public launch)
- Tests (write them after the pilot validates the model; cost > benefit at this stage)

Acceptance criteria

Before declaring "done":

1. I can visit `/` and see the Clio listing with photos and €15,800.
2. I can visit `/afiliado`, sign up, and immediately see my unique URL on the next page.
3. Visiting `/?ref={my_code}` sets the cookie. Submitting `/interesse` creates a lead linked to my affiliate record.
4. The buyer receives a WhatsApp with their code (or, if `WHATSAPP_ENABLED=false`, the message is logged to console).

5. The owner receives a WhatsApp with the new lead (same fallback).
 6. I can log into `/admin` with the password, see the lead, change its status to `closed_won`, and see "€500 owed to {affiliate}".
 7. Deployed to a public Vercel URL with all env vars set.
-

Draft PT-PT copy

Landing page hero

Renault Clio V — 2022 Particular. Sem comissões. Sem intermediários a inflar o preço.
€15.800 [Tenho interesse]

Below the fold (trust + affiliate hint)

Este anúncio é gerido diretamente pelo proprietário. Se chegou aqui através de um link de alguém conhecido, traz consigo um código de desconto de €300 ao submeter o seu contacto.

`/interesse` form intro

Deixe os seus dados. O proprietário entra em contacto por WhatsApp no próprio dia. Os seus dados são usados apenas para esta venda.

`/afiliado` intro

Tem alguém à procura de um carro? Inscreva-se e receba um link próprio. Por cada venda fechada a partir do seu link, recebe €500. Sem mensalidades, sem mínimos.

`/afiliado/[code]` dashboard

O seu link: `{BASE_URL}/?ref={code}` [Copiar] Comissão por venda fechada: €500
Pagamento por MB Way ou transferência após escritura.

Recommended build order (for the 3h window)

1. **0-20 min** — Bootstrap: `create-next-app`, Tailwind, shadcn/ui init, Supabase project + tables, env vars in `.env.local`, deploy hello-world to Vercel and confirm.
2. **20-60 min** — `/` LP (static, hardcoded) + ref cookie middleware + `/interesse` form (no WhatsApp yet, just persist).
3. **60-100 min** — `/afiliado` signup + `/afiliado/[code]` dashboard.
4. **100-140 min** — `/admin` with Basic Auth + leads table + status editor + affiliates table.
5. **140-170 min** — Wire WhatsApp module (or stub if not ready), confirmation page, copy polish.
6. **170-180 min** — Final deploy, smoke test all flows end-to-end on the production URL.

If you fall behind: cut the affiliate dashboard down to just "here's your link" with no lead list. That's the only flow that can shrink without breaking the pilot.

Legal note (do not skip)

For this pilot only, affiliates are personal-network and commissions will be paid informally. Before opening affiliate signup to the public, the owner must resolve: (a) whether each affiliate operates under recibos verdes (atividade aberta, Cat. B), (b) GDPR/RGPD compliance for the buyer data form (cookie banner, privacy policy URL, data retention statement), (c) tax treatment of the platform's role as an intermediary. These are platform-stage issues, not pilot-stage. Do not market the affiliate program publicly until they are resolved.